



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Crossplane vs Kubebuilder

paradigms, abstraction, complexity, extensibility and performance comparison with a common
use case implementation

Infrastructures for Cloud Computing and Big Data M
Lorenzo De Luca
0001240461

Kubernetes Operators



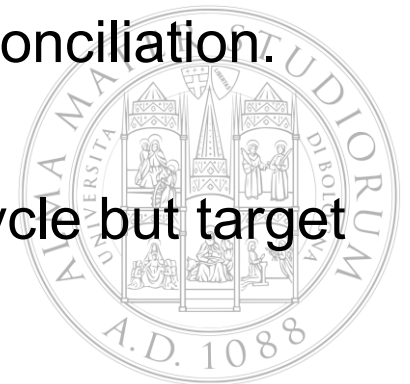
A Kubernetes Operator extends the Kubernetes API with a Custom Resource(CR) and a controller. Operators implement the reconciliation loop pattern: they observe Custom Resources through the Kubernetes API server, compare desired and actual state, and perform actions until convergence.

Building operators can be approached at different levels of abstraction:

Low-level: the developer writes all the controller logic

High-level: the developer describes the desired infrastructure API and composition logic, while existing controllers/providers perform reconciliation.

Both approaches solve the same problem of automating resource lifecycle but target different users and different tradeoffs.



Research Objective

Compare Crossplane and Kubebuilder as representatives of two paradigms for building Kubernetes Operators.

Will be evaluated:

- Architectural model and abstraction level
- Development complexity and extensibility
- Internal optimizations adopted by each framework
- Resource consumption

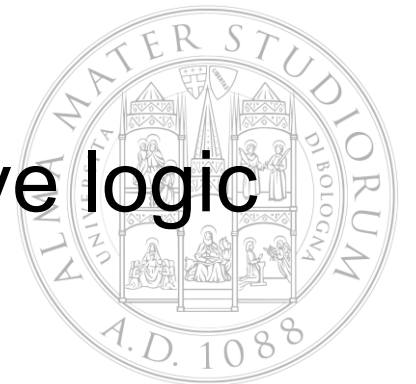
Does the convenience of the extra abstraction layer of Crossplane cost more in resources/latency than it saves in development effort, compared to a hand-written Kubebuilder controller?



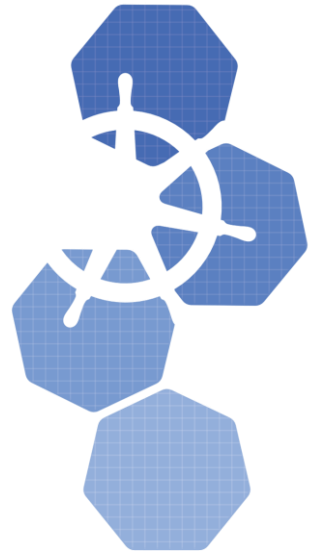
Crossplane



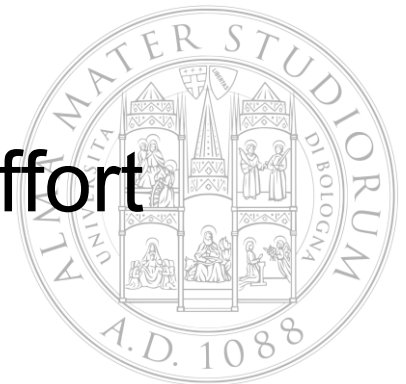
- Universal control plane
- Declarative composition model
- Strong for composition and platform engineering
- Logic is mostly declarative
- Strength: consistency and reuse
- Limitation: less flexible for complex imperative logic



Kubebuilder



- Low-level Go SDK
- Imperative control model
- Strong for writing specialized controllers from scratch
- Logic in Go
- Strength: maximum flexibility
- Limitation: higher coding and maintenance effort



Architecture

Crossplane

User applies a **Claim** or **Composite Resource**



Crossplane composition controller observes the resource



Composition is rendered into composed **Managed Resources**



Providers reconcile each Managed Resource against Kubernetes or external systems



Status aggregated bottom-up:
Provider -> MR -> XR → Claim

Kubebuilder

User applies a **Custom Resource**



Controller-runtime informer watches the API server



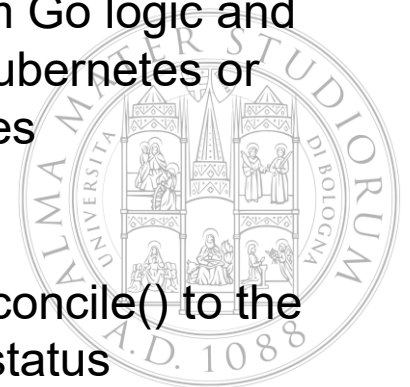
Events are added to a work queue



Reconcile() executes custom Go logic and **directly** creates/updates Kubernetes or external resources

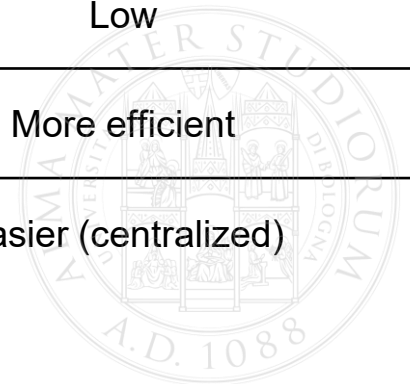


Status written directly by Reconcile() to the Custom Resource status



Features comparison

Aspect	Crossplane	Kubebuilder
Abstraction	High (declarative)	Low (imperative)
Architecture	Multi-layer	Single controller
Development effort	Lower	Higher
Extensibility	Providers, Functions, patching	Arbitrary code, any library
Flexibility	Medium	Very high
Reusability	High	Low
Performance	Higher overhead	More efficient
Debugging	Harder	Easier (centralized)



Optimizations

Crossplane:

- Uses Kubernetes controller patterns internally: informers, caches, work queues and rate limiting.
- Providers reconcile managed resources independently, enabling parallel and modular reconciliation.
- Leader election avoids duplicate reconciliation in replicated controller deployments.

Kubebuilder:

- Informer cache: most Kubernetes object reads are served from a local cache, reducing API-server load
- Uses a rate-limited work queue with backoff after repeated errors.
- Supports event predicates and *MaxConcurrentReconciles* for tuning reconciliation load.

So... Which one is better? It depends...

Scenario	Better choice	Reason
Standard infrastructure composition	Crossplane	Declarative and reusable
Internal Developer Platform	Crossplane	User-friendly claims and abstractions
Multi-resource self-service APIs	Crossplane	Composition model fits well
Complex conditional logic	Kubebuilder	Full control in Go
Custom external API integration	Kubebuilder	Any Go library can be used
Fine-grained lifecycle control	Kubebuilder	Reconciliation is fully programmable
Provisioning latency	Kubebuilder	Fewer controller layers
Debuggability	Kubebuilder	Less layers, more control

Kafka Topic Self-Service

Code and raw data obtained:

<https://github.com/lorenzodeluca/kafka-operator-clash>

Common goal



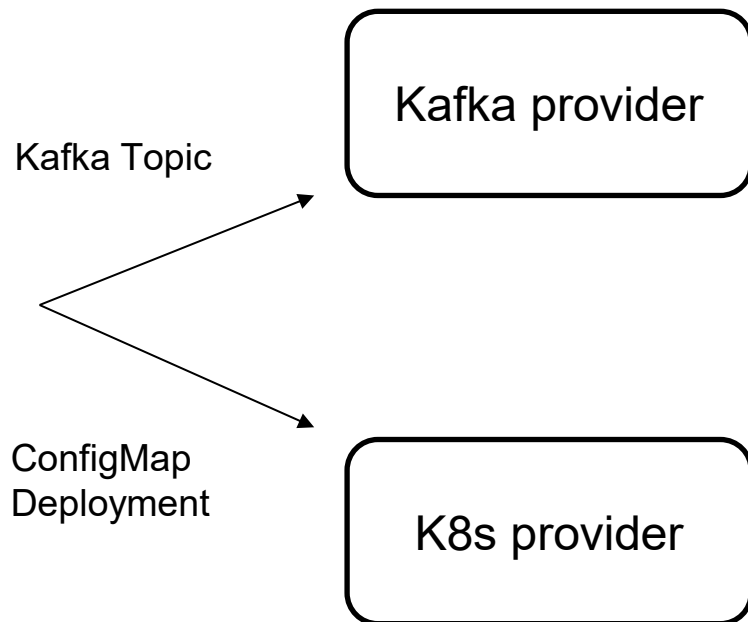
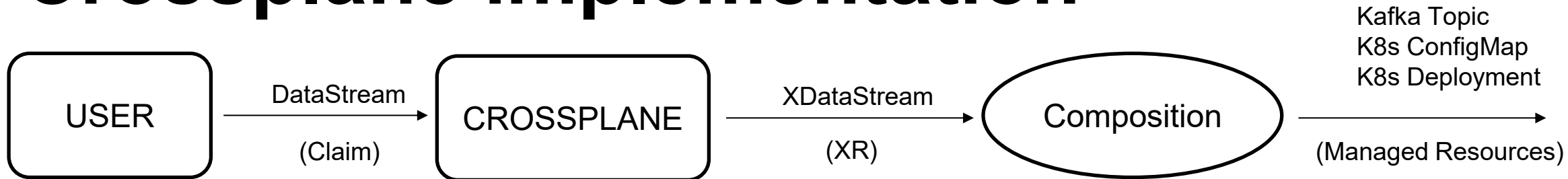
The goal for the implemented operators is to create a new Kafka topic. Instead of manually creating all required resources, the user declares a single object called *DataStream*.

When a *DataStream* is created, the operator must automatically provision and reconcile a ready-to-use data pipeline: a Kafka topic where data can flow, a configuration file with the connection details, and a small consumer application that reads the connection details generated by the operator.

DataStream example:

```
1  apiVersion: messaging.lorenzodeluca.it/v1alpha1
2  kind: DataStream
3  metadata:
4    name: lorenzo-sandbox-stream
5    namespace: default
6  spec:
7    topicName: "user-telemetry-data"
8    partitions: 2
9    replicationFactor: 1
```

Crossplane implementation



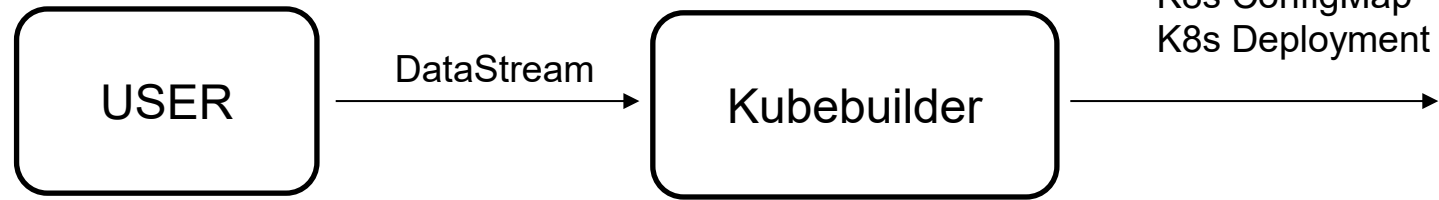
api/datastream_xrd.yaml:

```
1  apiVersion: apiextensions.crossplane.io/v1
2  kind: CompositeResourceDefinition
3  metadata:
4    name: xdatastreams.messaging.lorenzodeluca.it
5  spec:
6    group: messaging.lorenzodeluca.it
7    names:
8      kind: XDataStream
9      plural: xdatastreams
10   claimNames:
11     kind: DataStream
12     plural: datastreams
13   versions:
14     - name: v1alpha1
15       served: true
16       referenceable: true
17       schema:
18         openAPIV3Schema:
19           type: object
20           properties:
21             spec:
22               type: object
23               properties:
24                 topicName:
25                   type: string
26                 partitions:
27                   type: integer
28                   default: 1
29                 replicationFactor:
30                   type: integer
31                   default: 1
32               required:
33                 - topicName
```

compositions/datastream_composition.yaml

```
- step: patch-and-transform
  functionRef:
    name: function-patch-and-transform
  input:
    apiVersion: pt.fn.crossplane.io/v1beta1
    kind: Resources
    resources:
      - name: kafka-topic
        base:
          apiVersion: topic.kafka.crossplane.io/v1alpha1
          kind: Topic
          spec:
            providerConfigRef:
              name: kafka-default
            forProvider:
              partitions: 1
              replicationFactor: 1
        patches:
          - type: FromCompositeFieldPath
            fromFieldPath: spec.topicName
            toFieldPath: metadata.name
          - type: FromCompositeFieldPath
            fromFieldPath: spec.partitions
            toFieldPath: spec.forProvider.partitions
          - type: FromCompositeFieldPath
            fromFieldPath: spec.replicationFactor
            toFieldPath: spec.forProvider.replicationFactor
```

Kubebuilder implementation



datastream_types.go:

```
1 package v1alpha1
2
3 import (
4     metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
5     "k8s.io/apimachinery/pkg/runtime"
6 )
7
8 // DataStreamSpec defines the desired state of DataStream
9 type DataStreamSpec struct {
10     // TopicName is the name of the Kafka topic to manage.
11     // +kubebuilder:validation:MinLength=1
12     TopicName string `json:"topicName"`
13
14     // Partitions defines the total partition count for the topic.
15     // +kubebuilder:default:=1
16     // +kubebuilder:validation:Minimum=1
17     Partitions int32 `json:"partitions,omitEmpty"`
18
19     // ReplicationFactor defines the replication factor.
20     // +kubebuilder:default:=1
21     // +kubebuilder:validation:Minimum=1
22     ReplicationFactor int16 `json:"replicationFactor,omitEmpty"`
23 }
24
```

datastream_controller.go:

```
153 func createKafkaTopic(broker, topic string, partitions int32, reps int16) error {
154     conn, err := kafka.Dial("tcp", broker)
155     if err != nil {
156         return fmt.Errorf("dial broker failed: %w", err)
157     }
158     defer conn.Close()
159
160     controller, err := conn.Controller()
161     if err != nil {
162         return fmt.Errorf("cannot get kafka controller: %w", err)
163     }
164
165     controllerAddr := net.JoinHostPort(controller.Host, strconv.Itoa(controller.Port))
166     controllerConn, err := kafka.Dial("tcp", controllerAddr)
167     if err != nil {
168         controllerConn, err = kafka.Dial("tcp", broker)
169         if err != nil {
170             return fmt.Errorf("cannot dial kafka controller (%s) nor broker fallback (%s): %w", controllerAddr, broker, err)
171         }
172     }
173     defer controllerConn.Close()
174
175     err = controllerConn.CreateTopics(kafka.TopicConfig{
176         Topic:         topic,
177         NumPartitions: int(partitions),
178         ReplicationFactor: int(reps),
179     })
180     if err != nil {
181         l := strings.ToLower(err.Error())
182         if strings.Contains(l, "already exists") || strings.Contains(l, "topic already exists") {
183             return nil
184         }
185         return err
186     }
187     return nil
188 }
```

Performance results

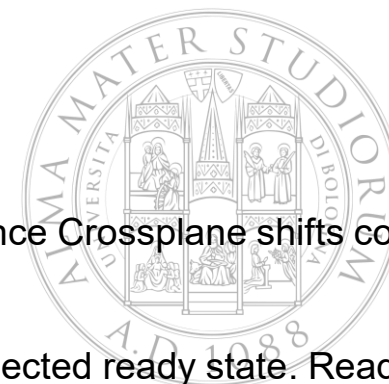
Metric	Crossplane	Kubebuilder
Lines of code written*	165	270
Manually edited files	7	2
Avg time to ready state (seconds)**	6	2
At rest CPU usage (millicores)	7	1.73
At rest RAM usage (MiB)	384	51.5

Environment:

- Debian VPS: 8 vCPU, 8 GB RAM, 75GB SSD
- Local k3s cluster
- Single Kafka broker
- Controllers/providers already running
- Crossplane components running in-cluster
- Kubebuilder controller running as host process with 'make run'
- Metrics collected through `kubectl top` for pods and '/proc' for Kubebuilder

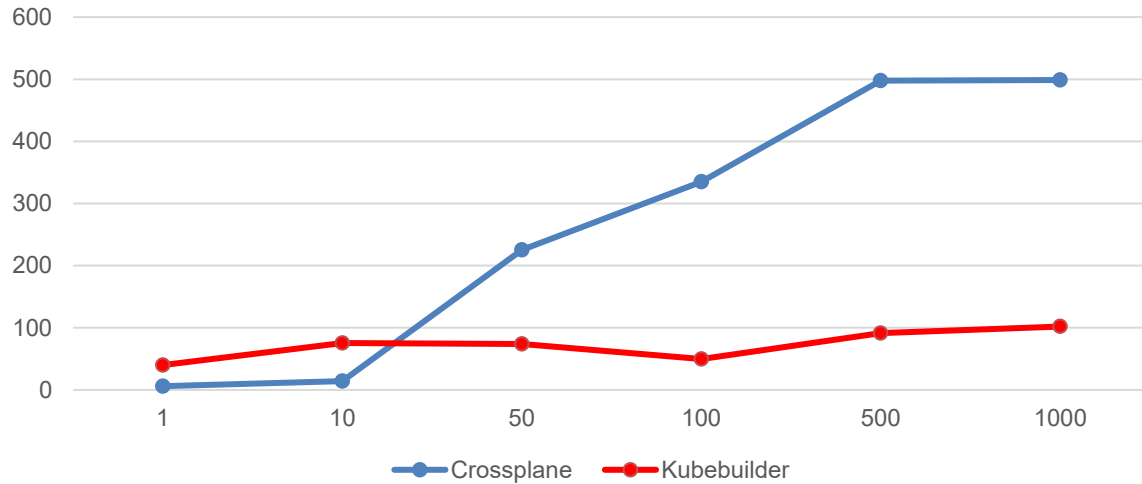
* Excluding generated code and provider installation/configuration manifests. The metric is indicative only since Crossplane shifts complexity to providers and platform configuration, while Kubebuilder concentrates complexity in the controller code.

** The measured time is the elapsed time between applying the DataStream resource and observing the expected ready state. Ready state means that the framework-reported DataStream/Claim is Ready and the expected Kafka topic, ConfigMap and Deployment are observable.

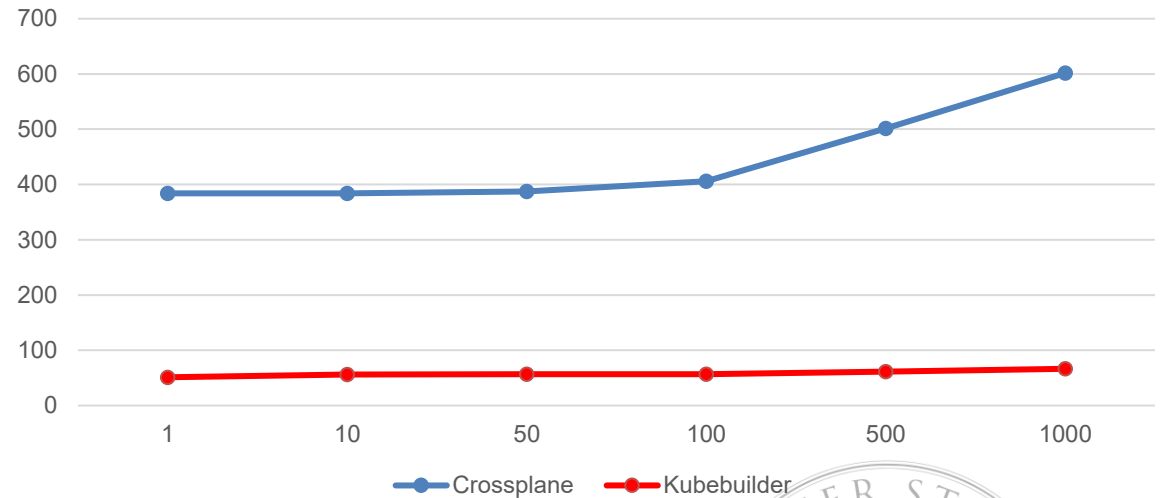


Resources consumption

Average CPU usage vs DataStream objects batch size



Average RAM usage vs DataStream objects batch size

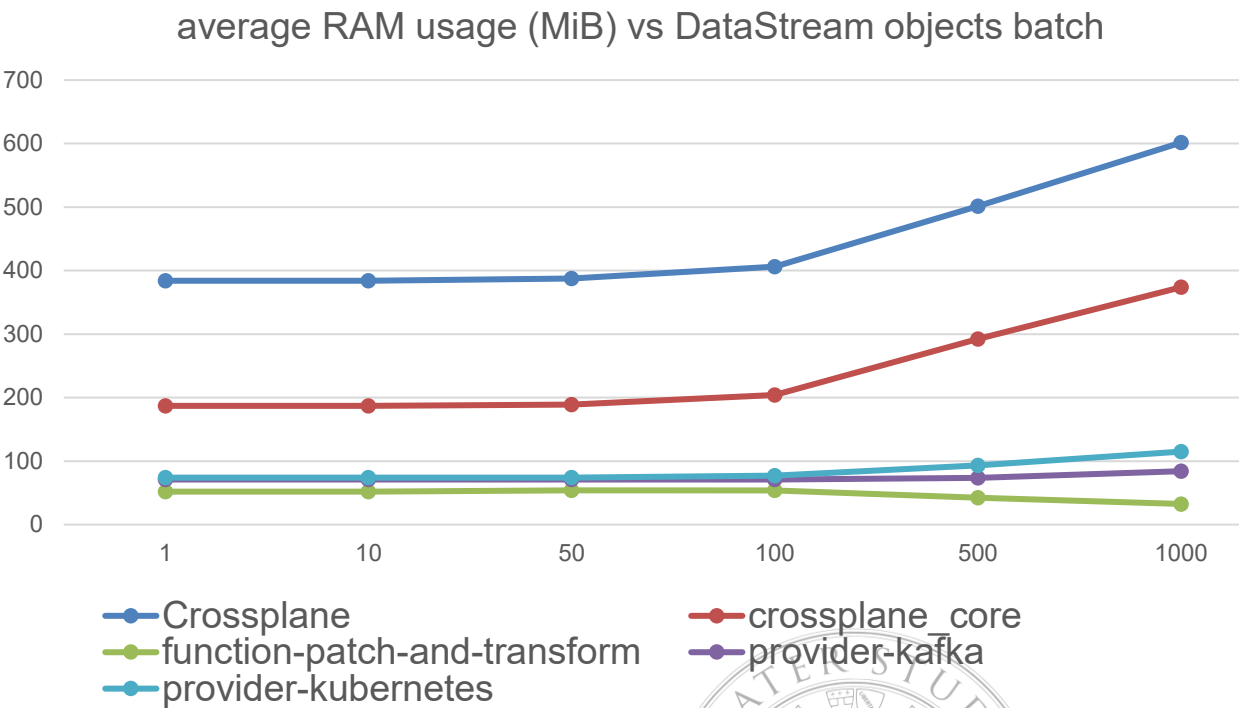
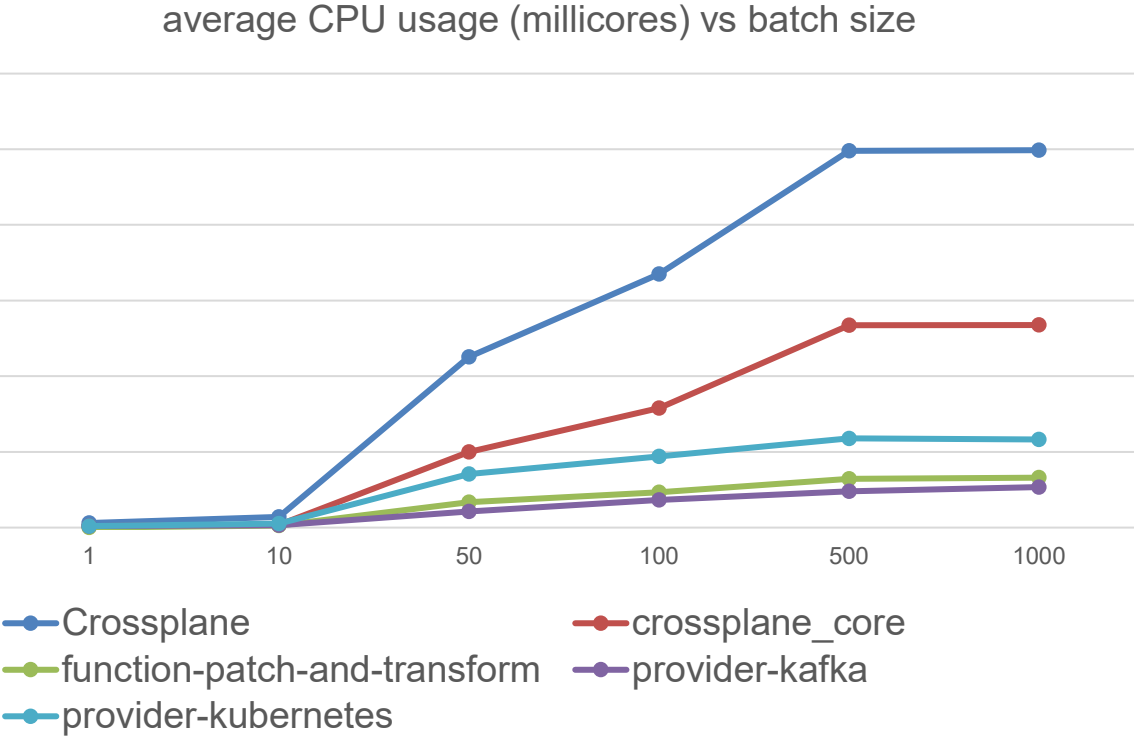


X-axis: batch size, the number of DataStream resources created together

Y-axis: average CPU rate (millicores)/RAM usage(MiB) each component sustained while it was actively reconciling that batch

*Crossplane measurements include the control-plane components used in the experiment: Crossplane core, provider-kafka, provider-kubernetes and function-patch-and-transform.

Resources: Crossplane components



X-axis: batch size, the number of DataStream resources created together
Y-axis: average CPU rate (millicores)/RAM usage(MiB) each component sustained while it was actively reconciling that batch



REFERENCES

- The Kubebuilder Book, <https://book.kubebuilder.io>
- Crossplane documentation, <https://docs.crossplane.io/v2.3>
- kafka-go, <https://github.com/segmentio/kafka-go>



Questions?

Thank you for your attention

